

QA পাইপলাইনে ফাস্ট টোকেনাইজার

এখন আমরা question-answering (প্রশ্নোত্তর) পাইপলাইনের ভিতরে গভীরে যাব এবং দেখব কীভাবে offsets ব্যবহার করে context থেকে প্রশ্নের উত্তর বের করা যায়—যেমনটি আমরা আগের সেকশনে grouped entities-এর ক্ষেত্রে করেছি। এরপর আমরা দেখব কীভাবে খুব লম্বা context (যা truncate হয়ে যায়) সামলানো যায়। যদি আপনি question answering টাস্কে আগ্রহী না হন, তাহলে এই অংশটি এড়িয়ে যেতে পারেন।

প্রশ্নোত্তর পাইপলাইন ব্যবহার করা

আমরা নিচের মতো করে question-answering পাইপলাইন ব্যবহার করতে পারি:

```
from transformers import pipeline
```

```
question_answerer = pipeline("question-answering")
```

```
context = ""
```

Transformers তিনটি জনপ্রিয় deep learning লাইব্রেরি—Jax, PyTorch এবং TensorFlow—দ্বারা সমর্থিত, এবং এদের মধ্যে সহজ ইন্টিগ্রেশন রয়েছে।

একটি লাইব্রেরিতে মডেল ট্রেন করে অন্যটিতে সহজেই inference করা যায়।

```
"""
```

```
question = "Transformers কোন কোন deep learning লাইব্রেরি দ্বারা সমর্থিত?"
```

```
question_answerer(question=question, context=context)
```

```
{'score': 0.97773,
```

```
'start': 78,
```

```
'end': 105,
```

```
'answer': 'Jax, PyTorch and TensorFlow'}
```

অন্যান্য পাইপলাইনের মতো নয়—যেগুলো মডেলের সর্বোচ্চ দৈর্ঘ্যের বেশি হলে টেক্সট truncate করে এবং ভাগ করতে পারে না (ফলে ডকুমেন্টের শেষে থাকা তথ্য মিস হতে পারে)—এই পাইপলাইন খুব লম্বা context সামলাতে পারে এবং প্রশ্নের উত্তর ডকুমেন্টের শেষেও থাকলেও সেটি খুঁজে বের করতে পারে।

```
long_context = ""
```

Transformers: আধুনিক NLP-এর অবস্থা

Transformers হাজার হাজার pretrained মডেল সরবরাহ করে, যা classification, information extraction, question answering, summarization, translation, text generation সহ আরও অনেক কাজ করতে পারে, ১০০টিরও বেশি ভাষায়।

এর লক্ষ্য হলো উন্নত NLP সবাইকে সহজভাবে ব্যবহার করতে দেওয়া।

Transformers API প্রদান করে, যাতে দ্রুত pretrained মডেল ডাউনলোড করে ব্যবহার করা যায়, নিজের ডেটাসেটে fine-tune করা যায় এবং পরে model hub-এ শেয়ার করা যায়।

কেন Transformers ব্যবহার করব?

1. সহজে ব্যবহারযোগ্য আধুনিক মডেল:
 - NLU এবং NLG টাস্কে উচ্চ পারফরম্যান্স
 - শিক্ষার্থী ও প্র্যাকটিশনারদের জন্য সহজ প্রবেশ
 - খুব কম abstraction, মাত্র তিনটি ক্লাস শিখলেই হয়
 - সব pretrained মডেলের জন্য একক API
 - কম কম্পিউট খরচ ও কম কার্বন ফুটপ্রিন্ট
2. গবেষকরা ট্রেন করা মডেল শেয়ার করতে পারেন:
 - নতুন করে ট্রেনিং করার প্রয়োজন কমে
 - সময় ও খরচ সাশ্রয় হয়
 - ১০,০০০+ pretrained মডেল, ১০০+ ভাষায়
3. মডেলের জীবনের প্রতিটি ধাপে সঠিক framework বেছে নেওয়া:
 - মাত্র ৩ লাইনের কোডে মডেল ট্রেন করা যায়
 - TF2.0 ও PyTorch-এর মধ্যে সহজে পরিবর্তন
 - training, evaluation ও production-এর জন্য সেরা framework নির্বাচন
4. সহজে কাস্টমাইজ করা:
 - প্রতিটি architecture-এর জন্য উদাহরণ দেওয়া আছে
 - মডেলের ভেতরের অংশ সহজে দেখা যায়
 - লাইব্রেরি ছাড়াও মডেল ফাইল ব্যবহার করা যায়

Transformers তিনটি জনপ্রিয় deep learning লাইব্রেরি—Jax, PyTorch এবং TensorFlow—দ্বারা সমর্থিত।

```
question_answerer(question=question, context=long_context)
```

```
{'score': 0.97149,  
'start': 1892,  
'end': 1919,  
'answer': 'Jax, PyTorch and TensorFlow'}
```

প্রশ্নোত্তরের জন্য একটি মডেল ব্যবহার করা

অন্য যেকোনো পাইপলাইনের মতোই, আমরা প্রথমে ইনপুট tokenization করি, তারপর সেটি মডেলের মধ্যে পাঠাই। question-answering পাইপলাইনের জন্য ডিফল্টভাবে ব্যবহৃত checkpoint হলো `distilbert-base-cased-distilled-squad`। নামের মধ্যে থাকা “squad” অংশটি এসেছে সেই

dataset থেকে, যেটির উপর মডেলটি fine-tune করা হয়েছে। SQuAD dataset সম্পর্কে আমরা Chapter 7-এ আরও আলোচনা করব।

```
from transformers import AutoTokenizer, AutoModelForQuestionAnswering

model_checkpoint = "distilbert-base-cased-distilled-squad"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
model = AutoModelForQuestionAnswering.from_pretrained(model_checkpoint)

inputs = tokenizer(question, context, return_tensors="pt")
outputs = model(**inputs)
```

আমরা question এবং context-কে একটি pair হিসেবে tokenize করছি, যেখানে question আগে রয়েছে।

question এবং context tokenization-এর একটি উদাহরণ

question-answering মডেলগুলো আমরা এখন পর্যন্ত যে মডেলগুলো দেখেছি, সেগুলোর থেকে একটু ভিন্নভাবে কাজ করে। উপরের ছবির উদাহরণ অনুযায়ী, মডেলটিকে এমনভাবে train করা হয়েছে যাতে এটি উত্তর কোথা থেকে শুরু হয়েছে সেই token-এর index (এখানে 21) এবং কোথায় শেষ হয়েছে সেই token-এর index (এখানে 24) ভবিষ্যদ্বাণী করতে পারে।

এই কারণেই এই মডেলগুলো একটি logits tensor ফেরত দেয় না, বরং দুটি দেয়: একটি answer-এর শুরু token-এর logits-এর জন্য, এবং আরেকটি answer-এর শেষ token-এর logits-এর জন্য।

এখানে যেহেতু আমাদের কাছে 66 token-সম্বলিত একটি মাত্র input আছে, তাই আমরা পাই:

```
start_logits = outputs.start_logits
end_logits = outputs.end_logits
print(start_logits.shape, end_logits.shape)
```

```
torch.Size([1, 66]) torch.Size([1, 66])
```

এই logits-গুলোকে probability-তে রূপান্তর করতে আমরা softmax function ব্যবহার করব। কিন্তু তার আগে নিশ্চিত হতে হবে যে context-এর অংশ নয়—এমন index-গুলো mask করা হয়েছে।

আমাদের input-এর গঠন হলো:

[CLS] question [SEP] context [SEP]

তাই question-এর token-গুলো এবং [SEP] token-টিও mask করতে হবে। তবে [CLS] token-টিকে mask করব না, কারণ কিছু মডেল এটি ব্যবহার করে বোঝাতে যে উত্তরটি context-এর মধ্যে নেই।

যেহেতু পরে আমরা softmax ব্যবহার করব, তাই যেসব logits mask করতে চাই সেগুলোকে একটি বড় ঋণাত্মক সংখ্যায় প্রতিস্থাপন করলেই হবে। এখানে আমরা **-10000** ব্যবহার করছি:

```
import torch
```

```
sequence_ids = inputs.sequence_ids()
# context-এর token ছাড়া বাকি সব mask করো
mask = [i != 1 for i in sequence_ids]
# [CLS] token-টিকে unmask করো
mask[0] = False
mask = torch.tensor(mask)[None]
```

```
start_logits[mask] = -10000
end_logits[mask] = -10000
```

এখন যেহেতু আমরা অনাকাঙ্ক্ষিত অবস্থানগুলোর logits ঠিকভাবে mask করেছি, তাই softmax প্রয়োগ করা যায়:

```
start_probabilities = torch.nn.functional.softmax(start_logits, dim=-1)[0]
end_probabilities = torch.nn.functional.softmax(end_logits, dim=-1)[0]
```

এই পর্যায়ে চাইলে আমরা start এবং end probabilities-এর argmax নিতে পারতাম। কিন্তু তাতে এমন হতে পারে যে start index, end index-এর চেয়ে বড় হয়ে যায়। তাই আমাদের আরও সতর্ক হতে হবে।

আমরা সব সম্ভাব্য start_index এবং end_index-এর probability হিসাব করব, যেখানে start_index <= end_index। তারপর যে (start_index, end_index) জোড়ার probability সবচেয়ে বেশি হবে, সেটিই বেছে নেব।

ধরি, “উত্তরটি start_index-এ শুরু হয়েছে” এবং “উত্তরটি end_index-এ শেষ হয়েছে”—এই দুটি ঘটনা পরস্পর স্বাধীন। তাহলে উত্তরটি start_index-এ শুরু হয়ে end_index-এ শেষ হওয়ার probability হবে:

```
start_probabilities[start_index] * end_probabilities[end_index]
```

অতএব, সব score বের করতে আমাদের শুধু সব সম্ভাব্য গুণফল হিসাব করতে হবে যেখানে start_index <= end_index।

প্রথমে সব সম্ভাব্য product হিসাব করি:

```
scores = start_probabilities[:, None] * end_probabilities[None, :]
```

এরপর `start_index > end_index` যেসব ক্ষেত্রে, সেগুলোকে 0 করে mask করব। কারণ অন্য সব probability ধনাত্মক সংখ্যা।

`torch.triu()` ফাংশন 2D tensor-এর upper triangular অংশ ফেরত দেয়, তাই এটি আমাদের জন্য এই masking-এর কাজটি করবে:

```
scores = torch.triu(scores)
```

এখন আমাদের শুধু সর্বোচ্চ মানটির index বের করতে হবে। যেহেতু PyTorch flattened tensor-এর index ফেরত দেয়, তাই `//` এবং `%` ব্যবহার করে `start_index` ও `end_index` বের করতে হবে:

```
max_index = scores.argmax().item()
start_index = max_index // scores.shape[1]
end_index = max_index % scores.shape[1]
print(scores[start_index, end_index])
```

এখনও পুরো কাজ শেষ হয়নি, তবে অন্তত আমরা ইতোমধ্যেই উত্তরের সঠিক score পেয়ে গেছি। আগের সেকশনের প্রথম ফলাফলের সঙ্গে মিলিয়ে দেখলে এটি যাচাই করতে পারবেন।

0.97773

অনুশীলন

সবচেয়ে সম্ভাব্য পাঁচটি উত্তরের জন্য start এবং end indices বের করুন।

এখন আমাদের কাছে answer-এর `start_index` এবং `end_index` token-এর হিসেবে আছে। তাই পরের ধাপে এগুলোকে context-এর character index-এ রূপান্তর করতে হবে। এখানেই offsets খুব কাজে লাগবে। আমরা এগুলো নিয়ে token classification task-এর মতোই ব্যবহার করতে পারি:

```
inputs_with_offsets = tokenizer(question, context, return_offsets_mapping=True)
offsets = inputs_with_offsets["offset_mapping"]
```

```
start_char, _ = offsets[start_index]
_, end_char = offsets[end_index]
answer = context[start_char:end_char]
```

এখন সবকিছু format করে final result তৈরি করলেই হয়:

```
result = {
    "answer": answer,
    "start": start_char,
```

```

        "end": end_char,
        "score": scores[start_index, end_index],
    }
    print(result)

```

```

{'answer': 'Jax, PyTorch and TensorFlow',
 'start': 78,
 'end': 105,
 'score': 0.97773}

```

দীর্ঘ context সামলানো

আগের উদাহরণে ব্যবহৃত question এবং long context যদি tokenize করতে যাই, তাহলে token-এর সংখ্যা question-answering পাইপলাইনের সর্বোচ্চ সীমা 384-এর চেয়ে বেশি হবে:

```

inputs = tokenizer(question, long_context)

print(len(inputs["input_ids"]))

```

461

তাই আমাদের ইনপুটকে সেই সর্বোচ্চ দৈর্ঘ্যে truncate করতে হবে। এটি করার কয়েকটি উপায় আছে, কিন্তু আমরা question-কে truncate করতে চাই না, শুধু context-কে করতে চাই। যেহেতু context হলো দ্বিতীয় sentence, তাই আমরা "only_second" truncation strategy ব্যবহার করব।

এখানে যে সমস্যাটি হয়, তা হলো প্রশ্নের উত্তর truncated context-এর মধ্যে নাও থাকতে পারে। যেমন এখানে আমরা এমন একটি প্রশ্ন নিয়েছি যার উত্তর context-এর শেষের দিকে আছে, আর truncate করার পরে সেই উত্তর আর উপস্থিত নেই:

```

inputs = tokenizer(question, long_context, max_length=384, truncation="only_second")

print(tokenizer.decode(inputs["input_ids"]))

```

""""

[CLS] Which deep learning libraries back [UNK] Transformers? [SEP] [UNK] Transformers : State of the Art NLP

[UNK] Transformers provides thousands of pretrained models to perform tasks on texts such as classification, information extraction,

question answering, summarization, translation, text generation and more in over 100 languages.

Its aim is to make cutting-edge NLP easier to use for everyone.

[UNK] Transformers provides APIs to quickly download and use those pretrained models on a given text, fine-tune them on your own datasets and

then share them with the community on our model hub. At the same time, each python module defining an architecture is fully standalone and

can be modified to enable quick research experiments.

Why should I use transformers?

1. Easy-to-use state-of-the-art models:

- High performance on NLU and NLG tasks.
- Low barrier to entry for educators and practitioners.
- Few user-facing abstractions with just three classes to learn.
- A unified API for using all our pretrained models.
- Lower compute costs, smaller carbon footprint:

2. Researchers can share trained models instead of always retraining.

- Practitioners can reduce compute time and production costs.

- Dozens of architectures with over 10,000 pretrained models, some in more than 100 languages.

3. Choose the right framework for every part of a model's lifetime:

- Train state-of-the-art models in 3 lines of code.
- Move a single model between TF2.0/PyTorch frameworks at will.
- Seamlessly pick the right framework for training, evaluation and production.

4. Easily customize a model or an example to your needs:

- We provide examples for each architecture to reproduce the results published by its original authors.

- Model internal [SEP]

""

এর মানে হলো, মডেলের জন্য সঠিক উত্তর বের করা কঠিন হয়ে যাবে।

এটি ঠিক করতে question-answering পাইপলাইন আমাদের context-কে ছোট ছোট chunk-এ ভাগ করার সুযোগ দেয়, যেখানে maximum length নির্দিষ্ট করা যায়। আর যেন context ভুল জায়গায় ভাগ হয়ে উত্তর হারিয়ে না যায়, সেজন্য chunk-গুলোর মধ্যে কিছু overlap-ও রাখা হয়।

আমরা tokenizer-কে এই কাজটি করতে দিতে পারি `return_overflowing_tokens=True` যোগ করে। আর overlap কতটুকু হবে, তা `stride` argument দিয়ে নির্ধারণ করা যায়। নিচে একটি ছোট sentence-এর উদাহরণ দেওয়া হলো:

```
sentence = "This sentence is not too long but we are going to split it anyway."
```

```
inputs = tokenizer(
```

```
    sentence, truncation=True, return_overflowing_tokens=True, max_length=6, stride=2
```

```
)
```

```
for ids in inputs["input_ids"]:
```

```
print(tokenizer.decode(ids))
```

'[CLS] This sentence is not [SEP]'

'[CLS] is not too long [SEP]'

'[CLS] too long but we [SEP]'

'[CLS] but we are going [SEP]'

'[CLS] are going to split [SEP]'

'[CLS] to split it anyway [SEP]'

'[CLS] it anyway. [SEP]'

এখানে দেখা যাচ্ছে, sentence-টি এমনভাবে chunk-এ ভাগ হয়েছে যাতে `inputs["input_ids"]`-এর প্রতিটি entry-তে সর্বোচ্চ 6টি token থাকে। শেষ entry-টিকে অন্যগুলোর সমান আকারের করতে চাইলে padding যোগ করতে হতো। আর প্রতিটি entry-র মধ্যে 2 token করে overlap রয়েছে।

এখন tokenization-এর ফলাফলটি একটু কাছ থেকে দেখি:

```
print(inputs.keys())
```

```
dict_keys(['input_ids', 'attention_mask', 'overflow_to_sample_mapping'])
```

প্রত্যাশামতো আমরা input IDs এবং attention mask পেয়েছি। শেষের key `overflow_to_sample_mapping` হলো একটি map, যা জানায় কোন ফলাফলটি কোন sentence থেকে এসেছে। এখানে 7টি result-ই এসেছে tokenizer-এ পাঠানো একমাত্র sentence থেকে:

```
print(inputs["overflow_to_sample_mapping"])
```

```
[0, 0, 0, 0, 0, 0, 0]
```

একাধিক sentence একসাথে tokenize করলে এটি আরও বেশি কাজে লাগে। যেমন:

```

sentences = [
    "This sentence is not too long but we are going to split it anyway.",
    "This sentence is shorter but will still get split.",
]

inputs = tokenizer(
    sentences, truncation=True, return_overflowing_tokens=True, max_length=6, stride=2
)

print(inputs["overflow_to_sample_mapping"])

```

ফলাফল হবে:

```
[0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1]
```

এর মানে হলো, প্রথম sentence আগের মতোই 7টি chunk-এ ভাগ হয়েছে, আর পরের 4টি chunk এসেছে দ্বিতীয় sentence থেকে।

এখন আবার আমাদের long context-এ ফিরে আসি। আগেই বলা হয়েছে, question-answering পাইপলাইন ডিফল্টভাবে `max_length=384` এবং `stride=128` ব্যবহার করে। এটি সেই পদ্ধতির সঙ্গে সামঞ্জস্যপূর্ণ যেভাবে মডেলটিকে fine-tune করা হয়েছে। পাইপলাইন কল করার সময় `max_seq_len` এবং `stride` argument দিয়ে এগুলো পরিবর্তন করা যায়।

তাই tokenize করার সময় আমরা এই parameter-গুলো ব্যবহার করব। এছাড়াও padding যোগ করব যাতে সব sample-এর দৈর্ঘ্য এক হয় এবং tensor বানানো যায়। সঙ্গে offsets-ও চাইব:

```

inputs = tokenizer(
    question,
    long_context,
    stride=128,
    max_length=384,
    padding="longest",

```

```
truncation="only_second",  
return_overflowing_tokens=True,  
return_offsets_mapping=True,  
)
```

এই `inputs`-এ থাকবে input IDs, attention masks, offsets, এবং `overflow_to_sample_mapping`।
যেহেতু শেষের দুটি model-এর parameter নয়, তাই tensor-এ রূপান্তরের আগে আমরা সেগুলো `pop` করে বের করে
নেব। `map`-টি এখানে দরকার নেই, তাই সেটি সংরক্ষণও করছি না:

```
_ = inputs.pop("overflow_to_sample_mapping")  
offsets = inputs.pop("offset_mapping")
```

```
inputs = inputs.convert_to_tensors("pt")  
print(inputs["input_ids"].shape)
```

```
torch.Size([2, 384])
```

আমাদের long context দুই ভাগে split হয়েছে। তাই মডেলের মধ্য দিয়ে গেলে আমরা দুই সেট start logits এবং end logits পাব:

```
outputs = model(**inputs)
```

```
start_logits = outputs.start_logits  
end_logits = outputs.end_logits  
print(start_logits.shape, end_logits.shape)
```

```
torch.Size([2, 384]) torch.Size([2, 384])
```

আগের মতোই softmax নেওয়ার আগে context-এর অংশ নয় এমন token-গুলো mask করব। পাশাপাশি attention mask যেগুলোকে padding token হিসেবে চিহ্নিত করেছে, সেগুলোও mask করব:

```
sequence_ids = inputs.sequence_ids()

# context-এর token ছাড়া বাকি সব mask করো
mask = [i != 1 for i in sequence_ids]

# [CLS] token-টিকে unmask করো
mask[0] = False

# সব [PAD] token mask করো
mask = torch.logical_or(torch.tensor(mask)[None], (inputs["attention_mask"] == 0))

start_logits[mask] = -10000
end_logits[mask] = -10000
```

তারপর softmax ব্যবহার করে logits-কে probability-তে রূপান্তর করা যায়:

```
start_probabilities = torch.nn.functional.softmax(start_logits, dim=-1)
end_probabilities = torch.nn.functional.softmax(end_logits, dim=-1)
```

পরবর্তী ধাপটি ছোট context-এর ক্ষেত্রে যা করেছিলাম তার মতোই, তবে এবার এটি দুইটি chunk-এর প্রতিটির জন্য করতে হবে। আমরা প্রতিটি সম্ভাব্য answer span-এর score হিসাব করব, তারপর সবচেয়ে ভালো score-ওয়ালা span বেছে নেব:

```
candidates = []

for start_probs, end_probs in zip(start_probabilities, end_probabilities):

    scores = start_probs[:, None] * end_probs[None, :]

    idx = torch.triu(scores).argmax().item()

    start_idx = idx // scores.shape[1]
```

```
end_idx = idx % scores.shape[1]

score = scores[start_idx, end_idx].item()

candidates.append((start_idx, end_idx, score))
```

```
print(candidates)
```

```
[(0, 18, 0.33867), (173, 184, 0.97149)]
```

এই দুইটি candidate হলো প্রতিটি chunk-এ মডেল যে সেরা উত্তর খুঁজে পেয়েছে। মডেলটি দ্বিতীয় অংশে সঠিক উত্তর থাকার ব্যাপারে অনেক বেশি আত্মবিশ্বাসী, যা একটি ভালো লক্ষণ।

এখন শুধু এই token span-গুলোকে context-এর character span-এ map করতে হবে। উত্তর পাওয়ার জন্য আসলে দ্বিতীয়টিই দরকার, তবে প্রথম chunk-এ মডেল কী বেছে নিয়েছে তা দেখাও আগ্রহের বিষয়।

অনুশীলন

উপরের কোড এমনভাবে পরিবর্তন করুন যাতে প্রতি chunk নয়, পুরো context জুড়ে সবচেয়ে সম্ভাব্য পাঁচটি উত্তরের score এবং span ফেরত দেয়।

আগে যে **offsets** নিয়েছিলাম, সেটি আসলে offsets-এর একটি list, যেখানে প্রতিটি chunk-এর জন্য আলাদা একটি list আছে:

```
for candidate, offset in zip(candidates, offsets):
```

```
    start_token, end_token, score = candidate
```

```
    start_char, _ = offset[start_token]
```

```
    _, end_char = offset[end_token]
```

```
    answer = long_context[start_char:end_char]
```

```
    result = {"answer": answer, "start": start_char, "end": end_char, "score": score}
```

```
    print(result)
```

```
{'answer': '\nTransformers: State of the Art NLP', 'start': 0, 'end': 37, 'score': 0.33867}
```

```
{'answer': 'Jax, PyTorch and TensorFlow', 'start': 1892, 'end': 1919, 'score': 0.97149}
```

প্রথম ফলাফলটি বাদ দিলে, আমরা long context-এর জন্য পাইপলাইনের মতো একই result পাই।